👋 Project Anton

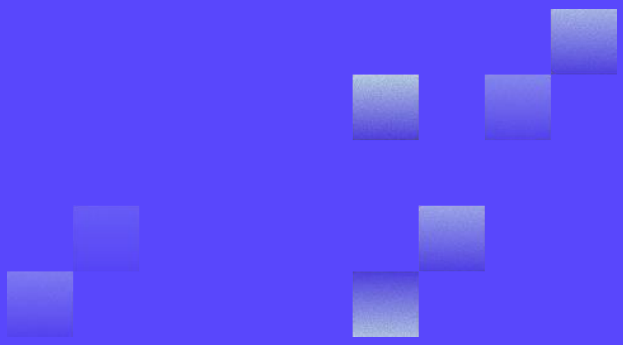
MERCOR

Project Anton | SWE Trajectory QC Annotation

This initiative creates and evaluates rigorous software-engineering tasks and the agent trajectories that attempt them. A task creator authors one complex task in Harbor format and generates at least two controlled agent rollouts. Two eligible rollouts are selected as Trajectory A and Trajectory B. Both selected trajectories must be rated PASS, PARTIAL, or FAIL. An annotator scores both trajectories, answers 8–15 task-specific binary rubrics, and submits one pairwise preference. A reviewer evaluates each annotation for correctness.

← To begin, please navigate through the project instructions on the left.

Welcome to **Project Anton | SWE Trajectory QC Annotation.**





NEW ONBOARDS

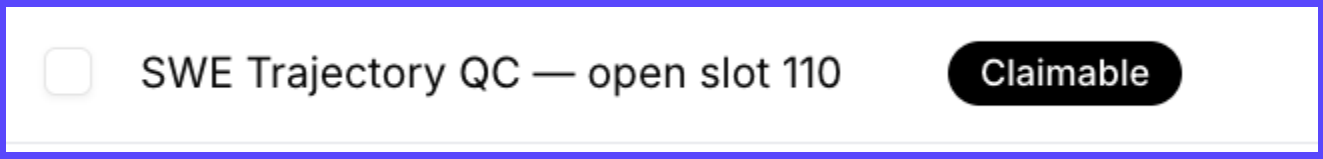
New Onboards	This document is for brand new experts who haven't done a task yet. Please review these instructions carefully.

- What time zone do you work out of?
- What times are you most active during that time zone?
- What days of the week do you plan to work?
- How many tasks would you like to complete by 08/28/2026?

After you have DM'd Donna, please also send a message in #anton-writers-bf1 to introduce yourself to the team. If you have any questions or issues with your first task, please share messages in #anton-general-bf1.

These tasks require careful software-engineering judgment and strong instruction following. Read the complete workflow before claiming or opening a task. Ask questions about missing task context, missing artifacts, or unclear requirements before investing substantial time.

Claim one SWE Trajectory QC task in Studio. A claimable task will be labelled as 'Claimable'

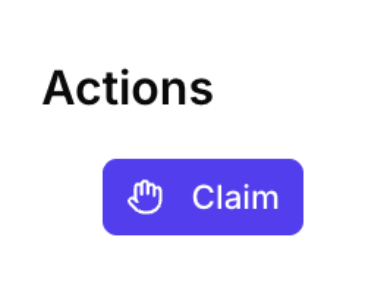


Please ONLY claim one task. If you do not own a task, do not resume someone else's work just and "click buttons" on the task. If we notice a repeated pattern of this, we will remove you from the project.

Claim the task promptly through the current Studio workflow. Confirm that the task package contains the task prompt, repository snapshot, environment instructions, test suite, and any reference artifacts supplied for the task.

If required task materials are missing or inaccessible, raise the issue in the approved project channel before starting the annotation.

Once you complete the task workflow, submit the required annotation artifacts and await review.



The Task Creator authors and validates one task, generates rollout candidates, selects eligible Trajectories A and B, and freezes the complete evaluation package. Each scores both trajectories, completes the task-specific rubrics, calculates final scores, and selects a pairwise preference without viewing the other Annotator’s work.

The Reviewer evaluates each annotation independently for correctness, completeness, evidence quality, and schema compliance. The Reviewer does not manually calculate or adjudicate IAA.

Task Workflow

General Task Workflow

This document describes the workflow for creating a rigorous SWE task, generating or obtaining one agent trajectory, independently evaluating the same task and trajectory across two annotators, and submitting evidence-based annotations for reviewer QC.

- **Open the task bundle in Studio** and read everything: the prompt as a contract, the repository, the environment, the test suite including hidden tests, and the golden patch where present.
- **Never modify the task.** The prompt, repo, tests and environment are frozen inputs. If something is broken, for example the Docker image will not build, report it in Review Feedback and escalate. Do not fix it, and do not score errored runs.
- **Know what the verifier enforces** before scoring: read the tests so no hidden requirement surprises you when judging the trajectories.

- **Press Generate Rollouts.** Studio produces Trajectory A and Trajectory B under identical frozen conditions, at least one of them Gemini.
- **Watch the first minutes.** A run that dies in seconds with a system or image-build error is not a trajectory. Retry once if transient; otherwise report and wait. Never annotate a run that is never executed.
- **Eligibility gate:** at least one trajectory must earn PASS or PARTIAL. Both trajectories failing means the pair cannot support rubric authoring, so regenerate or escalate.

8–15 binary YES/NO rubrics, written before scoring either side. Each must be objectively verifiable, framed so YES is always good, mapped to one of the 8 dimensions, and derived from the prompt, the environment, or senior engineering common knowledge. Never invented.

- **Balance ~50/50:** roughly half artifact rubrics (Dimensions 2 to 4, what the code looks like) and half process rubrics (Dimensions 5 to 8, what the agent did: files read, bug reproduced, tests run, honest summary).
- **Answer every rubric for both sides** with a one-line evidence reference each (file, turn, test name, command output).

For A, then B: read the complete trajectory, inspect the final patch separately from the process, compare the agent's claims with actual output, then score. Do not pick the preference before both sides are scored.

- **Dimension 0 is seeded and read-only.** PASS: reward = 1, every test passed. PARTIAL: 90 percent or more of tests passed. FAIL: under 90 percent. It is the completeness floor, not part of the numeric score. A FAIL paired with a PASS makes the comparison trivial, which is accepted. Verify the seeded value matches the run and report any mismatch.
- **Dimensions 1–8:** one integer 1–5 per dimension per trajectory, each with a written reason citing concrete evidence. The dimensions: 1 Correctness · 2 Architecture and trade-offs · 3 Conventions and diff discipline · 4 Robustness and API stability · 5 Instruction following and scope · 6 Planning, research and tools · 7 Verification and testing · 8 Communication and reporting.

Score	General meaning
5	Excellent, complete, senior-level execution with no meaningful defect
4	Strong execution with only minor inconsequential issues
3	Adequate and functional with noticeable but non-critical weaknesses
2	Major weaknesses, incomplete reasoning, fragile behavior, or material repair needed
1	Fundamentally broken, unsafe, deceptive, or unusable

Zero tolerance: test weakening, deletion, disabled assertions, or hardcoded expected results score 1 on Dimension 7.

- **Assign final quality scores** for A and B, consistent with the dimension scores.
- **The preference follows the gap:** equal scores = TIE, a one-point gap = A_BETTER or B_BETTER, two or more = MUCH_BETTER. The higher-scored trajectory must be the preferred one; a mismatch is flagged as a defect.
- **Write the comparative rationale:** where the sides agreed, where they differed, and why the winner's advantages outweigh the loser's. Ground it in the scores and rubric answers, not vibes.
- **Run AutoQC and submit.** Fix what it flags. The reviewer tab is read-only for annotators; never fill or fabricate reviewer outcomes.

Dimension 0 is an eligibility gate and is not included in the numeric final score.

- **PASS**: Complete or sufficiently complete. The solution works as intended by the task creator, with at most minor non-functional issues.
- **PARTIAL**: Core functionality works as intended, but one or two minor non-blocking edge cases or optional requirement was missed OR Usable after some user modification in non-execution based env.
- **FAIL**: Fundamentally incomplete or broken. Failed to achieve the core functional goal.

If both trajectories are FAIL, or the comparator is FAIL while Gemini passes, stop and escalate the pair-selection error.

Assign one integer from 1 to 5 for every dimension and trajectory. Higher is better.

Score	General meaning
5	Excellent, complete, senior-level execution with no meaningful defect
4	Strong execution with only minor inconsequential issues

3	Adequate and functional with noticeable but non-critical weaknesses
2	Major weaknesses, incomplete reasoning, fragile behavior, or material repair needed
1	Fundamentally broken, unsafe, deceptive, or unusable

Evaluate stated behavior, edge cases, regressions, and non-execution-based requirements.

- **5:** Fully correct; all material deterministic and qualitative requirements are satisfied.
- **4:** Correct on the intended path with one minor inconsequential issue.
- **3:** Core behavior works, but a noticeable non-blocking gap remains.
- **2:** Major requirements are missing or substantial user repair is needed.
- **1:** Core behavior is incorrect, deceptive, or unusable.

Evaluate structure, abstraction level, algorithm/data-structure choices, and context-appropriate trade-offs.

- **5:** Senior-level design, clean boundaries, sound trade-offs, and appropriate extensibility.
- **4:** Strong design with only minor imperfections.
- **3:** Functional but somewhat ad hoc, inefficient, over-engineered, or under-designed.
- **2:** Excessive coupling, poor abstraction, naive critical-path strategy, or mismatched design.
- **1:** Severely broken architecture, circular dependencies, irrational design, or extreme over-engineering.

Evaluate project idioms, helper reuse, duplication, surgical scope, and cleanup.

- **5:** Maintainer-quality code, project infrastructure reused, surgical diff, and no debug residue.
- **4:** Conventions followed and major utilities reused with minimal noise.
- **3:** Minor inconsistency, duplication, or unnecessary churn.
- **2:** Frequent convention violations, substantial reinvention, or a noisy patch.
- **1:** Repository conventions ignored, core infrastructure duplicated, or unrelated files changed without justification.

Evaluate failure semantics, malformed inputs, resources, concurrency, security, and compatibility.

- **5:** Comprehensive safeguards, appropriate failure behavior, and preserved APIs.
- **4:** Strong handling of expected failures and compatibility requirements.
- **3:** Primary cases handled, but rare edge conditions or awkward compatibility remain.

- **2:** Fragile behavior, masked errors, unsafe exception handling, or unplanned breaking changes.
- **1:** Severe bugs, deadlocks, leaks, vulnerabilities, silent corruption, or unhandled API breakage.

Evaluate explicit requirements, negative constraints, scope boundaries, and workflow rules.

- **5:** Every material instruction followed with no scope creep.
- **4:** All important boundaries followed with at most one benign addition.
- **3:** Core requirements met, but a minor instruction is missed or scope is slightly stretched.
- **2:** A material boundary or negative instruction is violated locally.
- **1:** Core constraints are blatantly ignored or severe unauthorized scope is introduced.

Evaluate decomposition, navigation, solution-space exploration, right-sized research, tools, and adaptation.

- **5:** Clear dependency-aware plan, right-sized exploration, precise tools, and effective adaptation.
- **4:** Clear plan, systematic navigation, effective tools, and consideration of the primary alternative.
- **3:** Correct area reached, but avoidable detours or premature commitment occur.
- **2:** Little planning, chaotic navigation, excessive research, premature telescoping, or repeated failed strategy.
- **1:** Aimless loops, blind edits, repeated rereading, or inability to make progress.

Expect rapid convergence on localized tasks and broader solution-space exploration on complex architectural tasks.

Evaluate verification evidence, edge-case coverage, test integrity, debug hygiene, and honesty.

- **5:** Reproduction created when applicable; happy paths and edge cases verified; focused tests added; results reported accurately; no debug residue.
- **4:** Appropriate functional verification and sound tests with no weakening.
- **3:** Provided verification used adequately, but little additional edge-case coverage is added.
- **2:** Weak checks, missed obvious cases, debug artifacts, or inadequately explained failures.
- **1:** No meaningful verification, failing output ignored, tests weakened/deleted, assertions disabled, or expected results hardcoded.

Test weakening is a zero-tolerance violation.

Evaluate intermediate communication and the final summary.

- **5:** Risks explained before execution; concise milestone updates; blockers surfaced early; final summary accurately reports changes, tests, rationale, and limitations.
- **4:** Risks and progress communicated clearly; final summary accurate and concise.
- **3:** Adequate communication and factual summary with minor omissions.
- **2:** Poor risk communication, unexplained silence, repetitive chatter, or vague final summary.
- **1:** Silence during destructive operations, concealed blockers, hallucinated verification, or deceptive/absent final summary.

---

6. Phase 5: Pairwise Evaluation & JSON Annotation Reporting

After scoring Part 1, Part 2, and Part 3 across all dimensions and evaluating the 5–10 binary prompt-specific rubrics for both Trajectory A and Trajectory B, annotators must compare them side-by-side and submit the final evaluation report in **JSON format**.

6.1 Overall Preference Scale

Select exactly one preference rating comparing Trajectory A to Trajectory B:

- **A_MUCH_BETTER**: Trajectory A is vastly superior in functional correctness, code elegance, or verification discipline; B failed or introduced severe issues.
- **A_BETTER**: Both completed the task, but A demonstrated superior trade-offs, cleaner code, or right-sized research.
- **TIE**: Both trajectories performed equally well (or equally poorly) across qualitative and functional dimensions.
- **B_BETTER**: Trajectory B demonstrated superior engineering quality or process discipline.
- **B_MUCH_BETTER**: Trajectory B is vastly superior; Trajectory A failed or exhibited severe flaws.

> **IMPORTANT Independent Reviewer (Task Auditor) Requirement:** To ensure high dataset integrity, an **Independent Reviewer (Task Auditor)** must review and approve the task definition, baseline completeness, and rubrics before final annotation package submission.

7.1 Independent Reviewer Audit Checklist

The Independent Reviewer must verify and sign off on the following:

Task Complexity & Non-Triviality: Confirm that the Harbor task challenges the agent beyond simple unit-test correctness (e.g., architecture, performance, conventions, safety).

Controlled Environment & Model Selection: Confirm that at least one trajectory used a Gemini model, and both executed under **exactly the same harness, prompt, and Harbor environment** (Section 3).

Part 1 Goal Completeness Parity: Confirm that both Trajectory A and Trajectory B achieve baseline Goal Completeness without dramatic discrepancy (no PASS vs FAIL pairs).

5–10 Binary Prompt-Specific Rubrics Validity: Confirm that 5–10 binary prompt-specific rubrics are defined, objectively verifiable, mapped to the 8 global dimensions, framed so YES is always good to have, and derived from legitimate sources (prompt, environment, or senior engineering common knowledge).

Over-Researching & Telescoping Audited: Confirm that the labeler properly penalized turns wasted on redundant searches, thrashing, or telescoping into an ad-hoc approach without scoping alternatives (Dimension 6).

Surgical Patch Audited: Confirm that the git diff was audited for extraneous reformatting, unsolicited refactoring, or dead code (Dimension 4).

Test Integrity Audited: Confirm that neither agent weakened or deleted existing tests to fake success (Dimension 7).

Final Score & Preference Consistency: Confirm that final_quality_score_a and final_quality_score_b are consistent with the pairwise preference (higher score = preferred trajectory).

JSON Schema Compliance: Confirm that the JSON report includes task_type, Part 1 status, "reason" fields for all prompt-specific rubrics and global dimensions, binary "answer_a"/"answer_b" (YES/NO) values, and a structured comparative rationale.

AutoQC should verify:

- Required files and valid JSON
- Task and trajectory identifier consistency
- Frozen prompt, harness, environment, and commit consistency
- At least one Gemini trajectory
- Both statuses in {PASS, PARTIAL}
- Exactly eight global dimensions with integer scores from 1 through 5
- Correct final-score arithmetic and two-decimal reporting

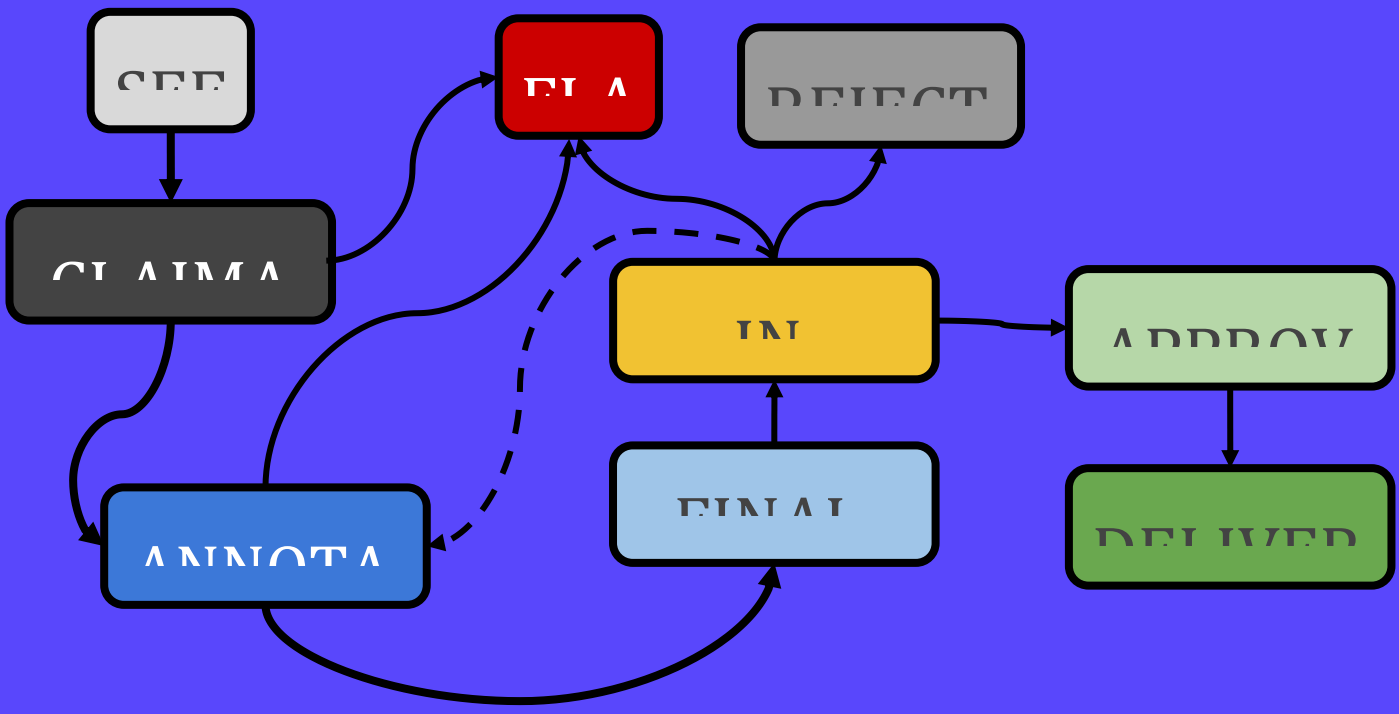
- Five to ten valid binary rubrics
- Valid task-format, task-category, source, answer, and preference enums
- Pairwise direction consistency
- Required reasons and evidence
- Reviewer outcome before delivery

After both reviewed annotations are complete, AutoQC appends IAA metadata. Reviewers do not edit this output. The team may manually audit a sample of completed tasks to verify that automated comparison remains reliable.

Mercor Studio Walkthrough

Studio Guide

Read below to walk through the studio workflow that is expected of you and find answers to common issues.

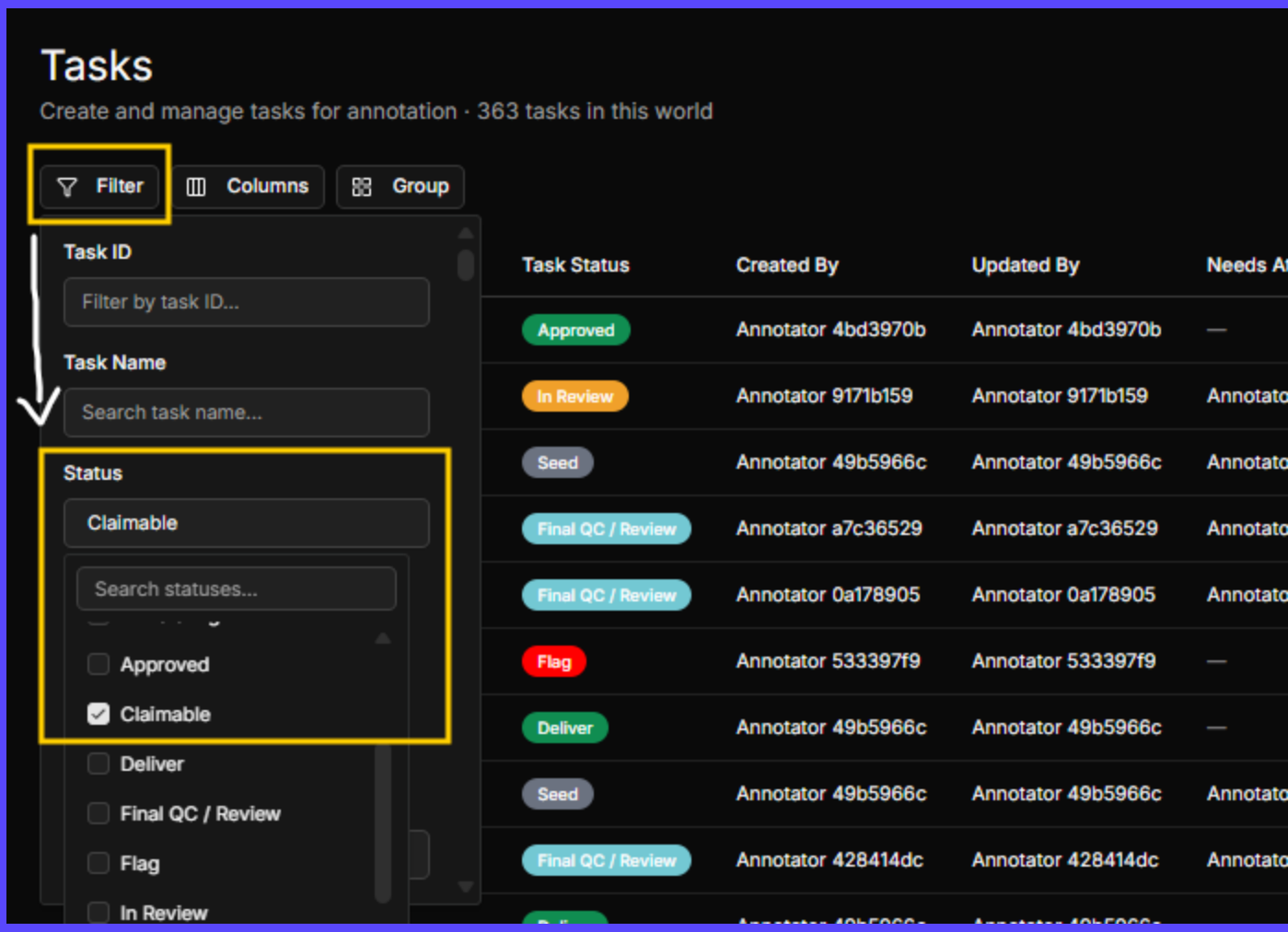


Annotators can find tasks to work on only in the **Claimable** status. Once submitted for review, a reviewer will pick up the task from **Final QC**. Any requested revisions will come back to the annotator in the **Annotating** status.

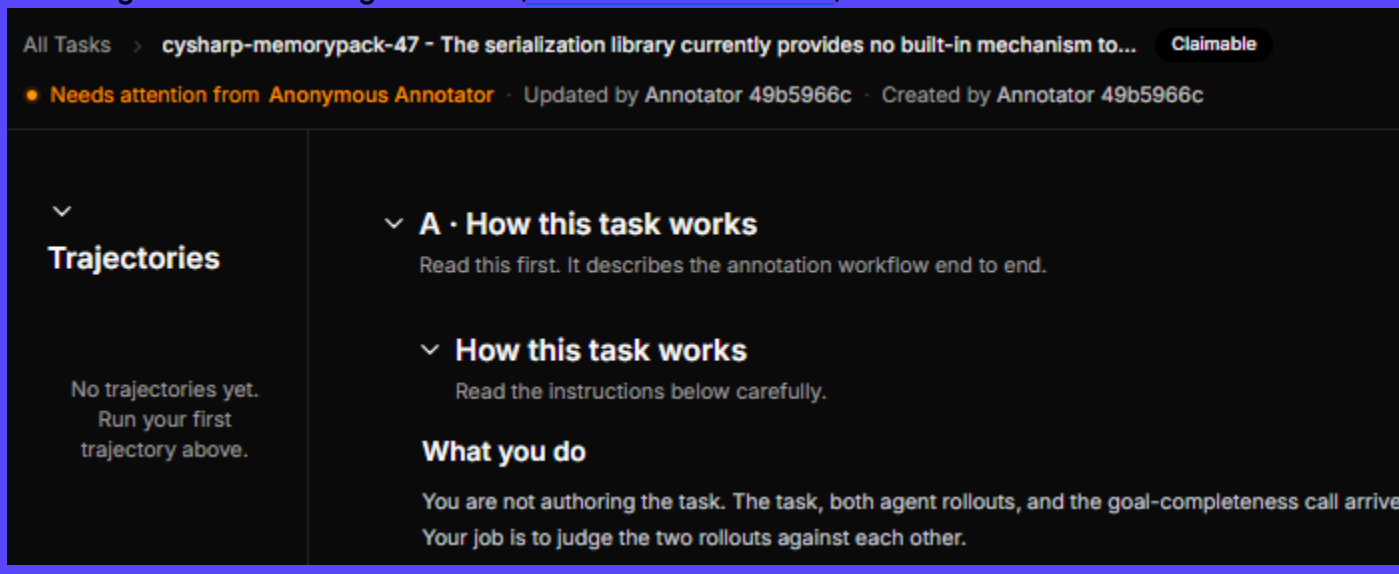
The 1-task limit applies to tasks owned by the annotator currently in the Annotating status (ie, either new tasks or tasks that need revisions).

1. Claim a task

Filter for "Claimable" task in **SWE Trajectory QC V2** or **Terminal Trajectory QC**



Click on an available task and once in the task detail screen, click **"Claim for annotating"** in the **Navigation** tab of the **right sidebar** (If there are no sidebars).



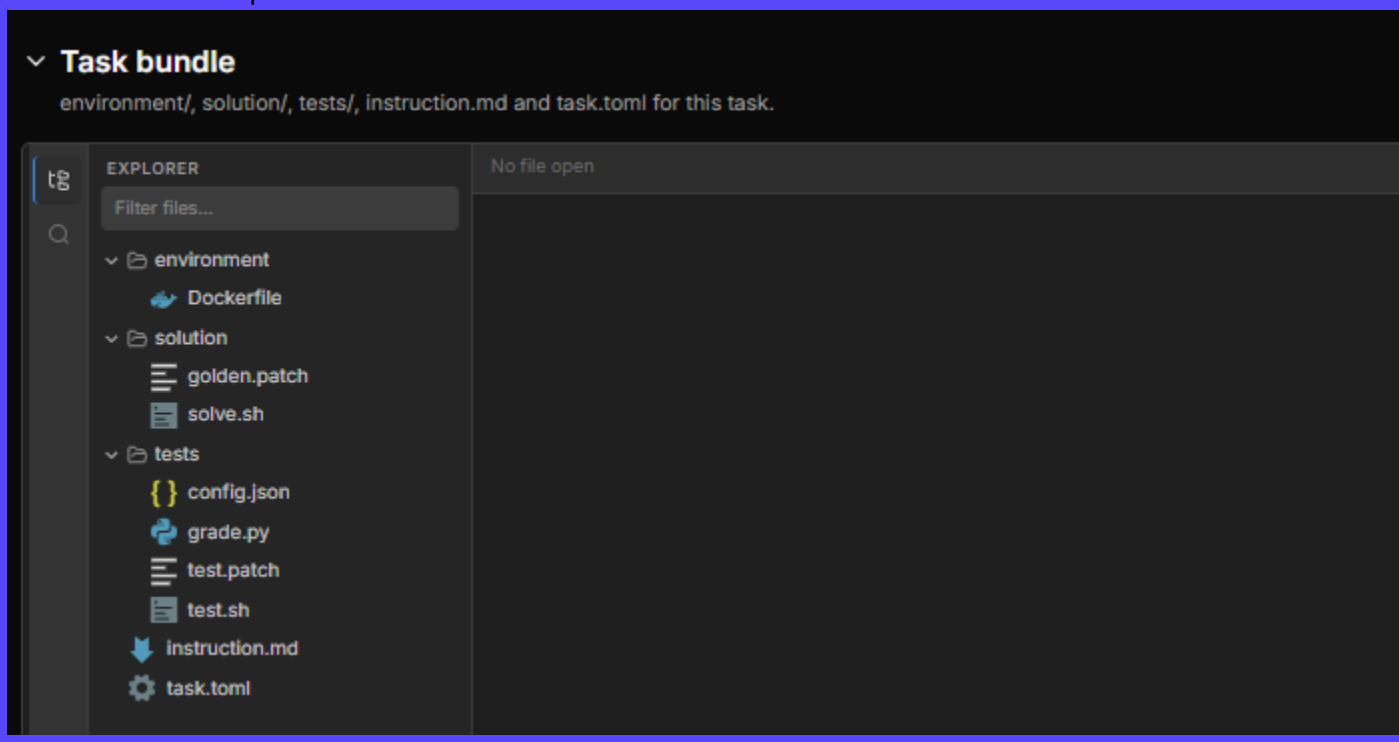
2. Understand the task

Read the task bundle

Tasks can come in a few different shapes. They are all valid so long as there are several key elements:

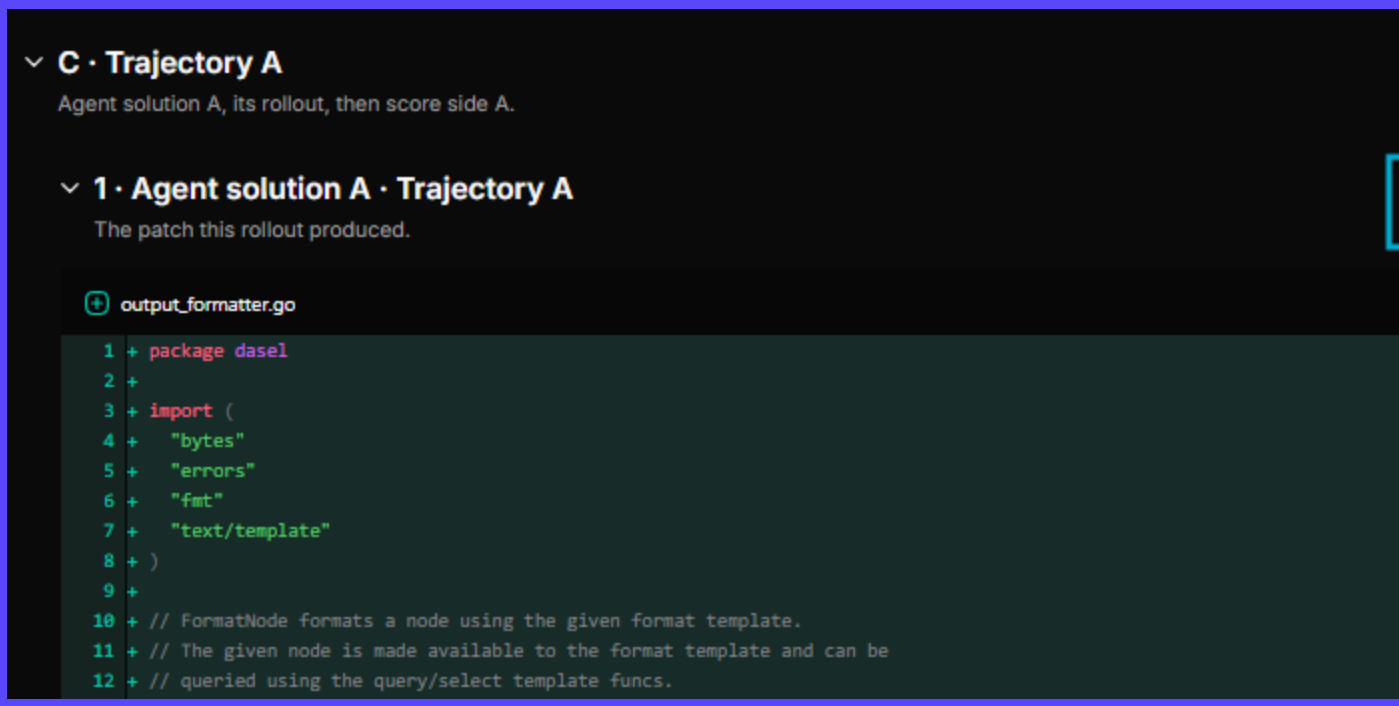
- Environment `Dockerfile`
- Hidden tests + `test.sh` script
- Prompt information (Can go by many names: `instruction.md`, `interface.md`, `description.md`, or `problem_statement.md`)

Task bundle example:

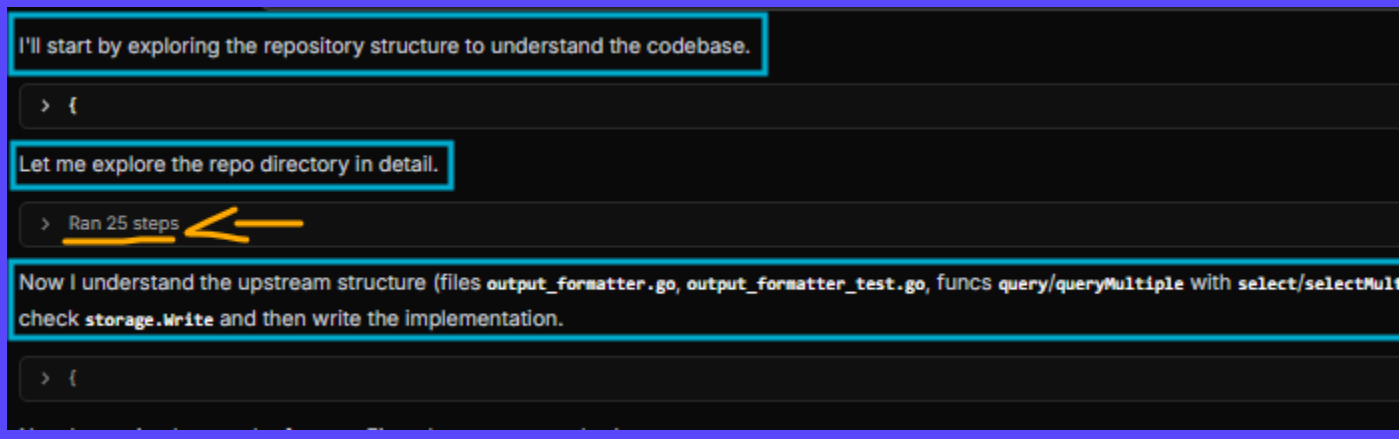


Scan the trajectories

Confirm there is a diff for each trajectory, and it is of reasonable size for the prompt requirements (eg, if the feature request is for a new API endpoint, the patch should not be solely test assets and a README). You can check the diff size in the upper right hand corner of the **Agent Solution** section. Compare this diff size with the agent's final summary or final git diff at the end of the trajectory to see if all of the files in the solution are present.



Each bold sentence is a turn taken by the agent. The agent will also make tool calls. Some agents make a single tool call per turn, others group many into one turn.

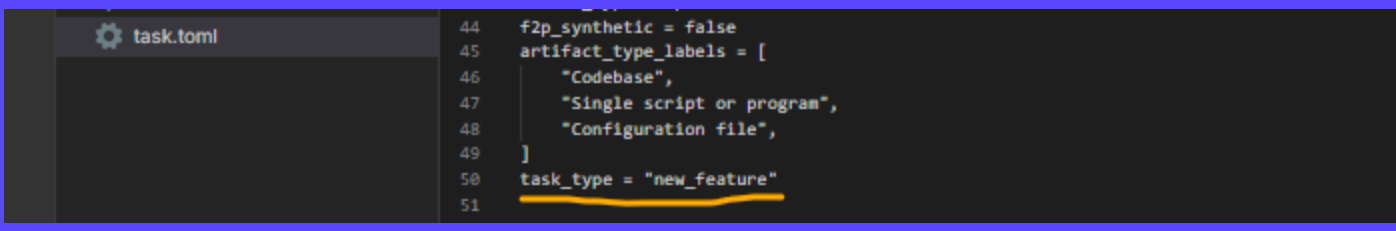


!!Before moving on:

1. Make sure you clearly understand the task requirements
2. If anything looks off, flag the task. (See also: When to Flag a Task)

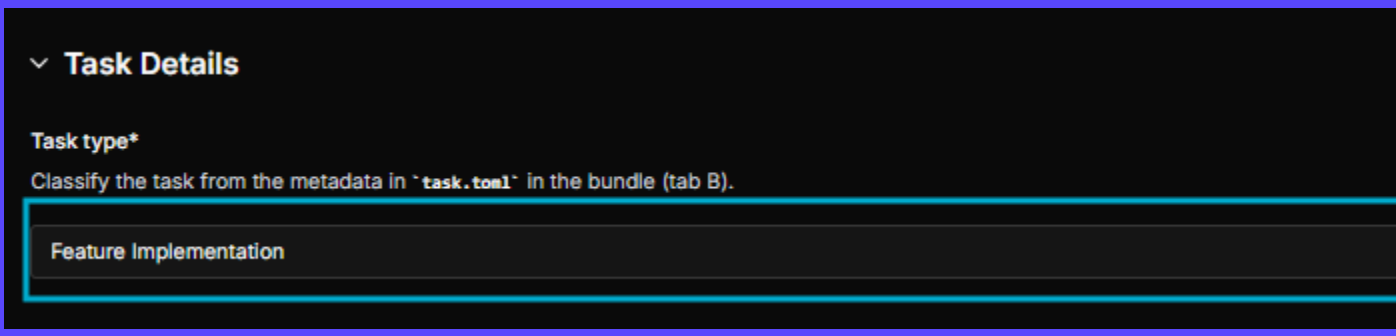
3. Assign a task type

If there is a `task.toml` file, use the `task_type` field value:



If there is no `task.toml`, decide on the closest type based on the prompt requirements:

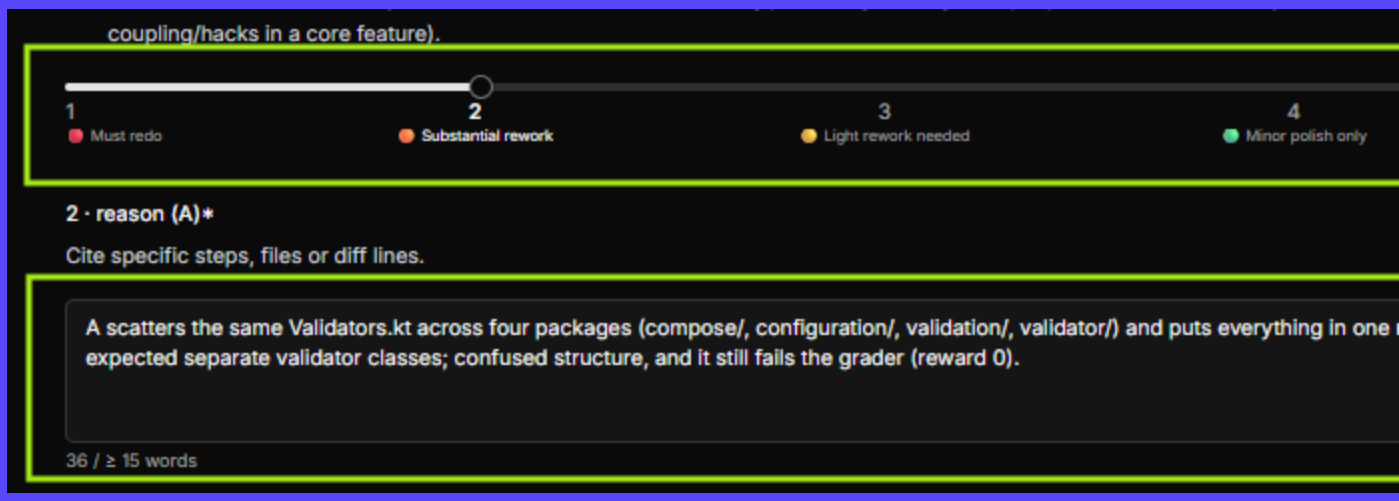
1. Feature Implementation
2. Bug Fix Deterministic
3. Bug Fix Nondeterministic
4. Refactor
5. Architecture Design
6. MVP
7. Codebase Migration
8. Performance Optimization
9. Test Writing



3. Score Rollouts A & B

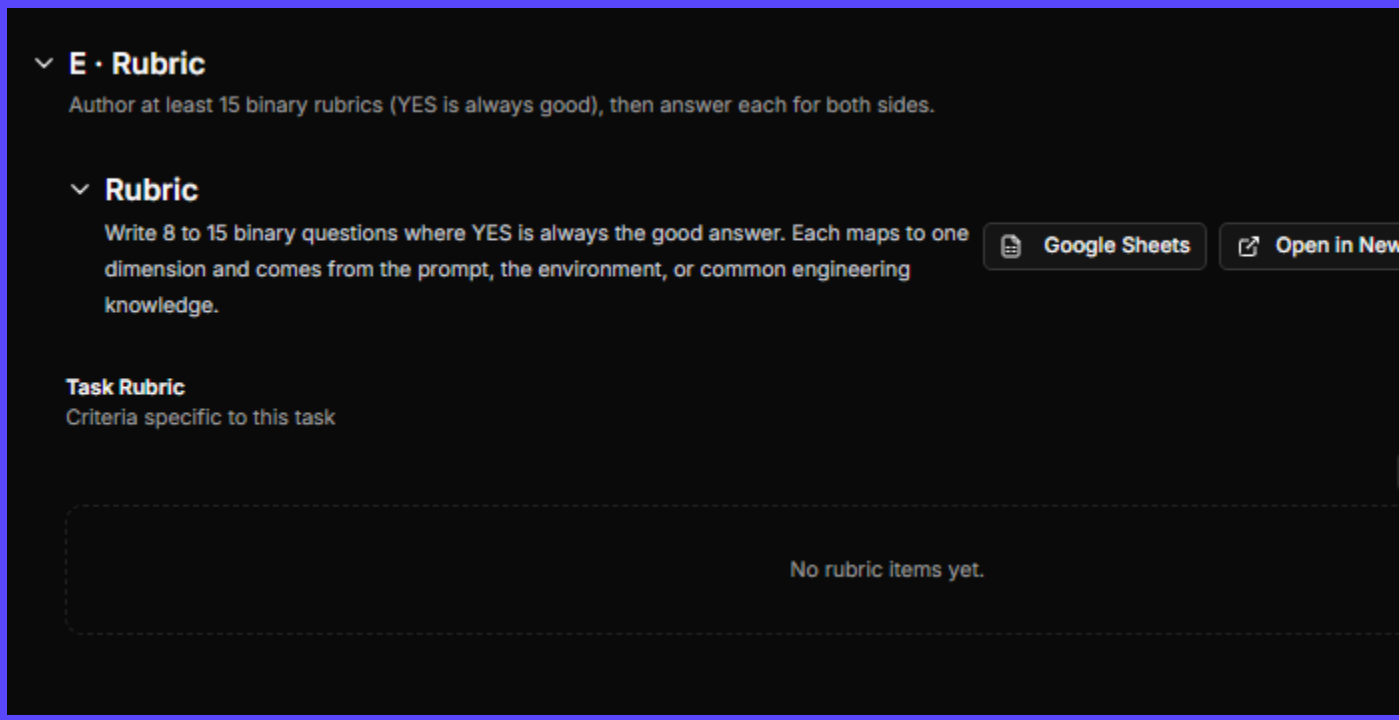
Inspect each trajectory and score it along the 8 dimensions.

Make sure each dimension has both a **score** (1-5) and a **reason** for the score. Cite specific evidence from the diff (include file names + line numbers) and/or specific step(s) in the trajectory.

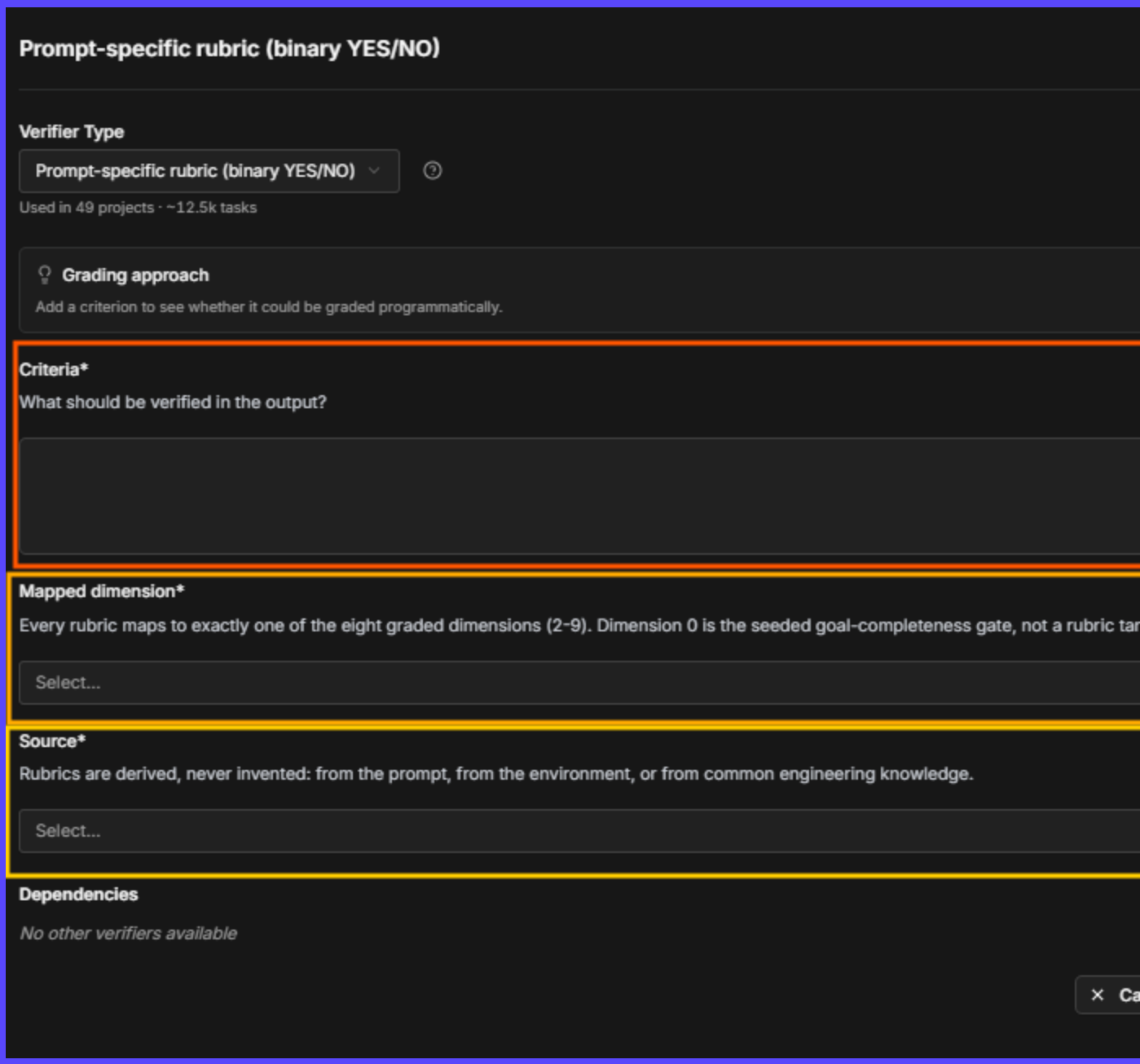


4. Write Rubric

Generate 8-15 rubric criteria based on the task requirements. (More information on rubrics.)



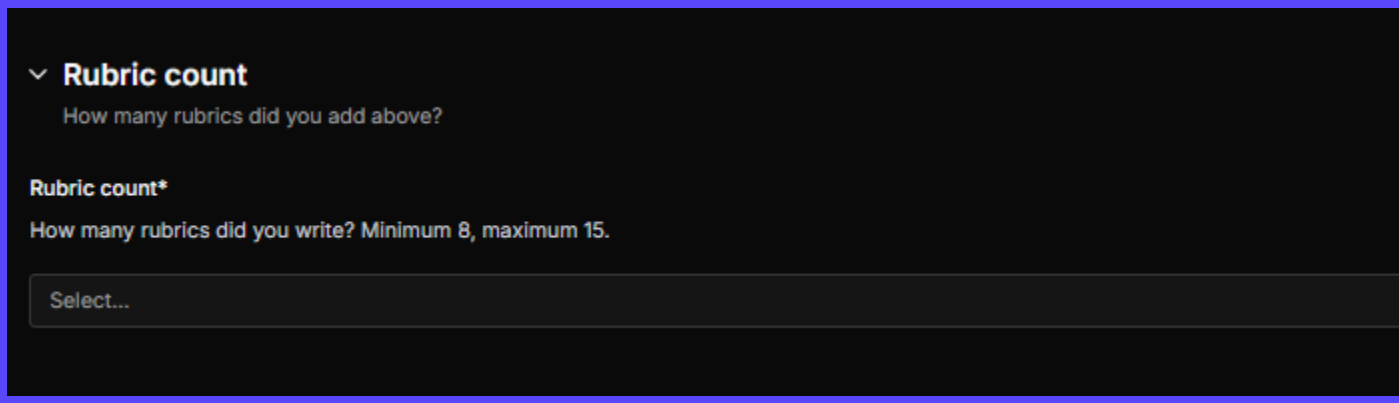
Click **Add Rubric Item** to add a new entry. Fill in the **Criteria**, **Mapped Dimension**, and **Source** fields then **Save** to create the rubric item.



Fill in 3 fields for each rubric item:

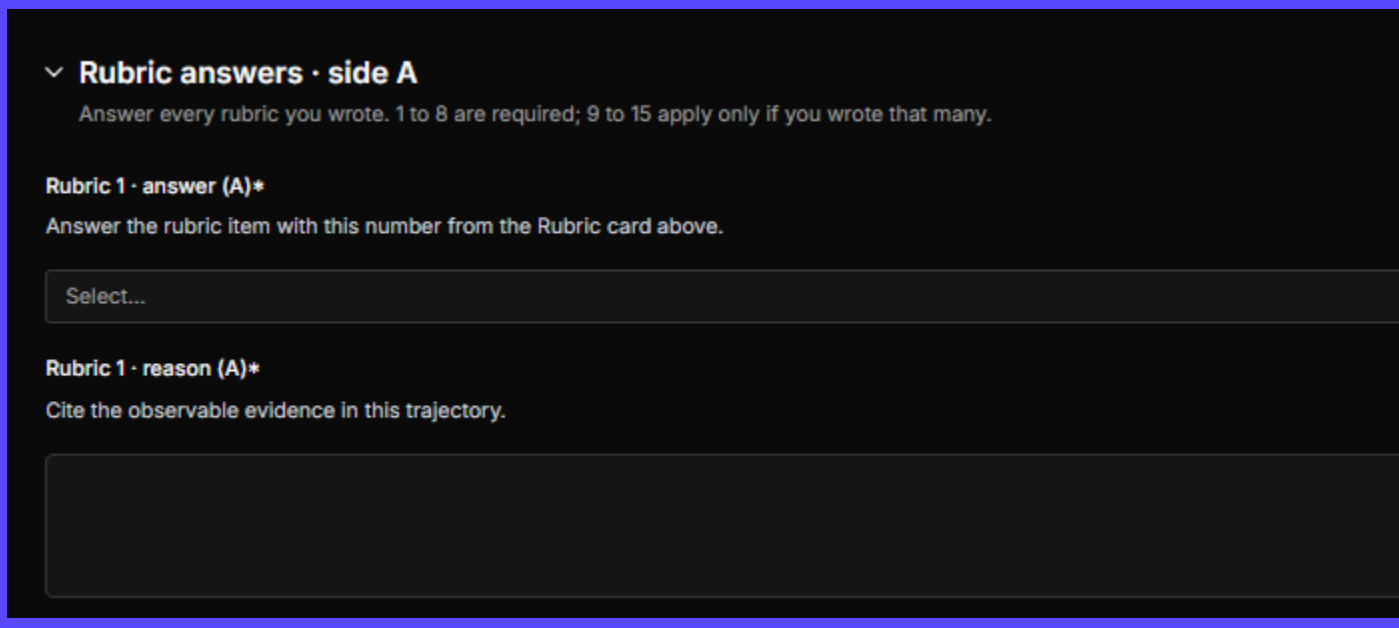
- **Criterion**: A binary (YES/NO) statement or question where YES is the good behavior. It should align with the requirements, and focus on behavior rather than specific outputs to avoid accidentally penalizing a different but still valid solution. (eg, “Agent removes all dead code” vs. “Agent deletes the unused `foo()` function”).
- **Mapped dimension:** One of the 8 dimensions
- **Source:** One of 3 options: Explicit prompt, inferrable from environment, or common engineering knowledge

You need at least 8 but no more than 15 rubric items. Prioritize clear, high-quality rubric items over volume. Once finished, fill in the rubric count field. This will generate the correct number of answer blocks for evaluating each trajectory. If you remove rubric criteria later, don't worry about excess evaluation blocks, just make sure the rationale box is clear and the **Rubric Count** value is accurate. YES/NO values cannot be cleared.



5. Score Rubric for A & B

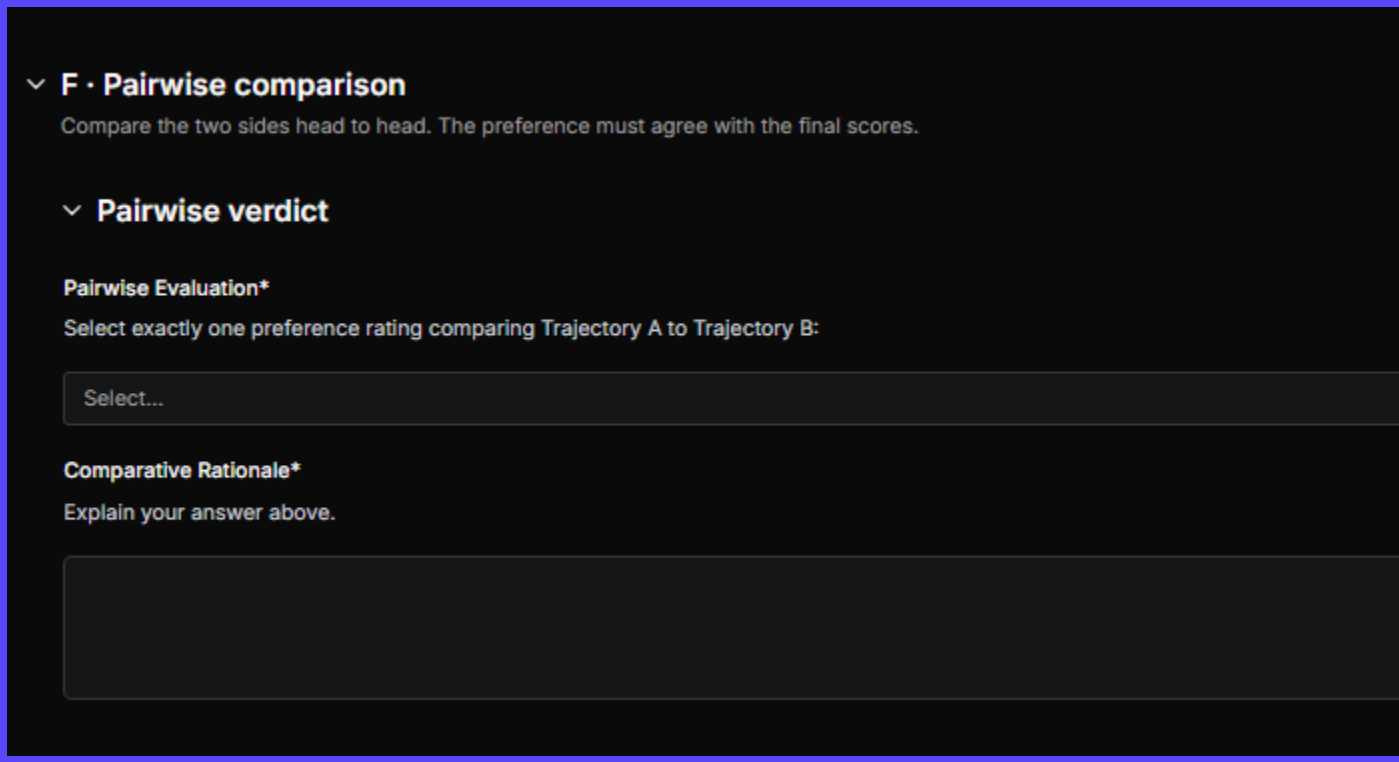
Evaluate each trajectory against all rubric criteria. Include both a YES/NO decision, and a reason citing specific evidence from the patch file or trajectory.



6. Pairwise comparison

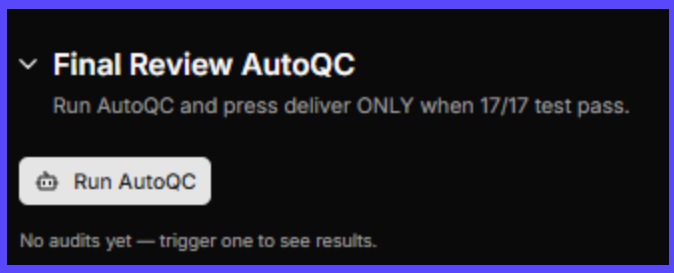
Complete the pairwise comparison of the two trajectories by making a final judgement and providing a rationale. Make sure your scoring on the dimensions and rubric line up with the final decision. If trajectory A is better, it should overall have a higher score than B.

- **A_MUCH_BETTER**: Trajectory A is vastly superior in functional correctness, code elegance, or verification discipline; B failed or introduced severe issues.
- **A_BETTER**: Both completed the task, but A demonstrated superior trade-offs, cleaner code, or right-sized research.
- **TIE**: Both trajectories performed equally well (or equally poorly) across qualitative and functional dimensions.
- **B_BETTER**: Trajectory B demonstrated superior engineering quality or process discipline.
- **B_MUCH_BETTER**: Trajectory B is vastly superior; Trajectory A failed or exhibited severe flaws.



7. AutoQC

Make sure AutoQC passes all checks or all non-passing checks have a valid dispute reason before submitting.



8. Submit

In order to submit, you must:

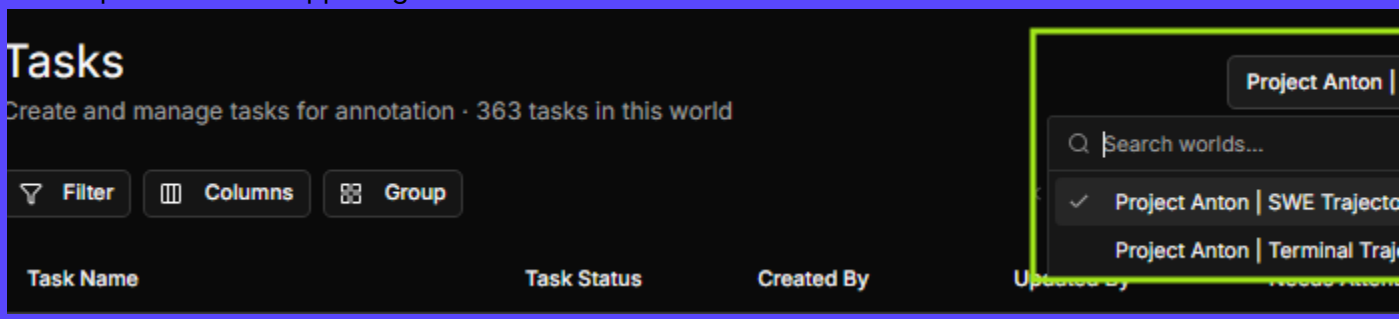
1. Have all green checks in the navigation pane list
2. A passing AutoQC run for the most recent version of the task

Once the task meets those requirements, press **Submit for Final QC**

9. Responding to feedback

If a reviewer has feedback, the task will come back to you in the Annotating stage with a “Needs attention from {YOUR NAME}” flag. You can find the reviewer’s comments in the **Reviewer Comments** tab of the right sidebar. If there are no comments, post the task link in **#anton-tooling-support-bf1**.

Tasks are currently available across two worlds. You can switch worlds by using the dropdown on the upper right hand corner of the main task view.



The easiest way to keep track of your tasks is with the task links, as tasks in both worlds may have similar names or task id numbers.

The task detail screen has three sections. At the midpoint of the screen is a square button with a **<** or **>**. This can be used to adjust the size or hide the sidebars.



Left sidebar (Green):
- Task navigation outline

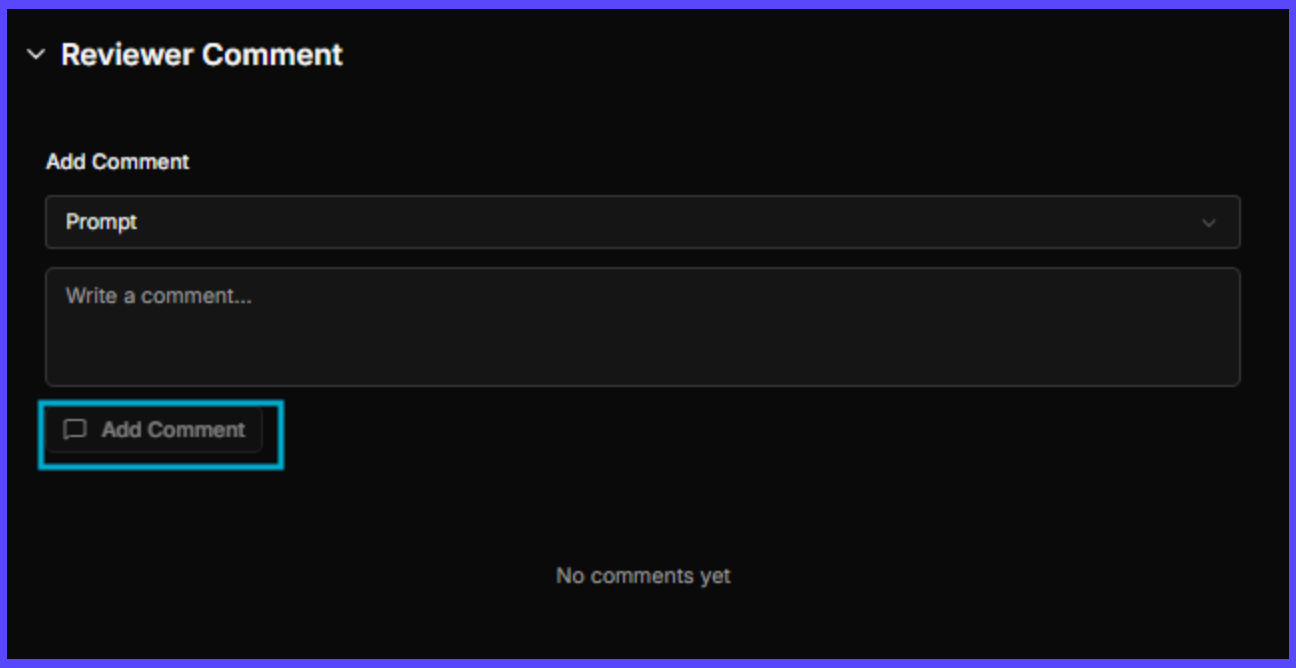
Main panel (Orange):
- Task contents and instructions (Sections A-F)
- Deliverable AutoQC (Must pass before submission)

Right sidebar (Blue):
- Navigation panel: Depending on the status, different buttons will appear here
    - Claimable: Claim for authoring, Flag Defective Task
    - Annotating: Submit for Final QC, Flag, Release
- Reviewer Comment: If a task is sent back for edits, the reviewer's comments will be here. Annotators can also leave comments.
- History: The claiming history of the task. Note: Reviewers and Admins also show up in the system as "Annotator 1a2b3cde5" and will show up in the history when a task is reviewed and marked for delivery.

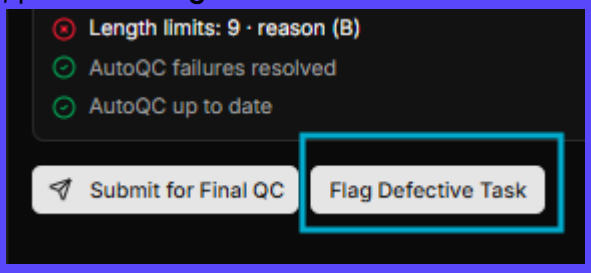
If a task is in the "Claimable" status and there is no button to claim it or you get an error message: Confirm you have no other tasks claimed or needing revisions (you can have 1 working task at a time).

If you do not have any active tasks and still cannot claim a task, post the task link in **#anton-tooling-support-bf1**.

1. **For reviewers only:** In the **Reviewer Comment** tab, leave a comment with the reason for flagging the task (don't worry about the subject dropdown), and click **Add Comment**



2. In the **Navigation** tab, press the **Flag Defective Task** button at the bottom of the pane



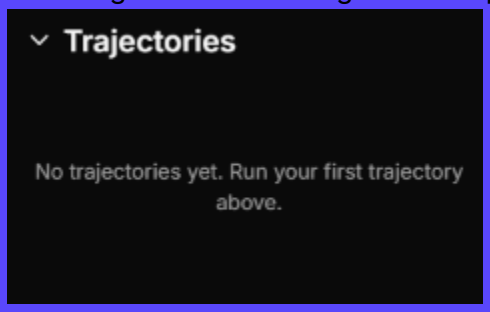
See also: How to Flag a Task

Issue	How to Spot It	Flag task?
Agent accessed outside network or otherwise found golden solution during trajectory	1. Check for *successful* `curl` or `pip install` commands for later versions of the task package or the github API. 2. Agents will also look for later commits or later versions of the package installed in the container environment	✅If the agent the task ❌If the agent Do not flag, but dimension 5 (In
Both agent solution sections are blank	Sections C.1 and D.1 do not show any file diffs	✅Flag the task
Only partial solution available	Sections C.1 and D.1 have files, but it is	✅Flag the task

for **both** agents	obvious it is not the entire solution (eg, a few test fixtures and a README as a solution to a new API endpoint)	
Solution is only missing for **one** agent, but the other one looks complete	One agent solution is missing some or all of the solution files, but the other agent produced a complete solution in the diff.	❌Do not flag t
Both trajectories fail	Both trajectories have `FAIL` on Dimension 0 result	❌Do not flag. scenario for ev

If your task is in the annotating stage and you are unable to edit any fields, post the task in **#anton-tooling-support-bf1**. Mercor Studio has intermittent permissioning issues and an admin will need to transition the task back into a state where you can edit it.

Please ignore this message at the top of the left sidebar:



Check if there are full trajectories under Section C.2 (C. Trajectory A > 2. Trajectory A) and D.2 (D. Trajectory B > 2. Trajectory B). If there are trajectories and patches there, then the task is fine.

For any issues not addressed here, post in **#anton-tooling-support-bf1**.

Make sure to include:
- The task link
- A short description of the issue

Golden Solution Guidance

Task and Trajectory Quality Guidance

This section defines the quality bar for a complete Trajectory QC package. A strong package contains a realistic SWE task, one complete shared trajectory, two independent evidence-based evaluations, and the artifacts required for reviewer QC and final JSON delivery.

A reviewer must be able to determine:

- What task was created and why it is sufficiently non-trivial
- Which repository snapshot and environment were used
- Which models and harness produced Trajectories A and B
- What each agent did during its trajectory
- What each final patch or artifact changed
- Which tests or evaluations were run for A and B
- Which evidence supports each rubric answer

Use specific references such as prompt excerpts, file paths, line numbers, test names, trajectory steps, patch hunks, command output, and final agent messages.

The annotation must distinguish task-design issues from agent-performance issues. A failed test may result from an incorrect patch, a broken environment, an underspecified task, or a brittle test. Inspect the underlying evidence before assigning a judgment.

The annotation must also distinguish a genuine task solution from a trajectory that passes visible tests by hardcoding, modifying tests, disabling assertions, or exploiting weak tests. Evaluate the actual behavior and engineering quality of each artifact.

Area	Strong package	Not acceptable
Task creation	Realistic SWE task requiring meaningful judgment	Artificial or trivial task with one obvious change
Shared task control	Both annotators receive the same frozen task package	Different prompts, commits, tests, or task versions
Trajectory capture	Complete commands, edits, patch, and final output	Informal summary without the actual trajectory
Verification	Reports actual commands, test outcomes, and logs	Claims success without output
Rubric evaluation	Cites trajectory, patch, and test evidence	Judges only from final pass/fail status
Independence	Annotators submit without coordinating judgments	Annotators share labels or rationale before submission
Submission	Includes all required artifacts and follows the current schema	Omits evidence or invents JSON fields

[EXT] Imperium | SWE Trajectory QC Annotation Instruction Document

Write concise, factual rationales. Explain what the task, trajectory, patch, and verification evidence show and why they support the selected rubric answer. Do not include generic commentary, unsupported certainty, repeated statements, or personal speculation.
Reviewers should never have to check whether your docstring, comments, or code contain slop; it must be understood by all parties that this is not allowed. Slop includes verbose generic comments, contradictory docstrings, stale references to reviewer feedback, fake certainty, irrelevant abstractions, and code that looks plausible but does not match the task.

The reviewer should be able to answer these questions quickly:

1. Is the created task complete, realistic, and sufficiently non-trivial?

2. Did both annotators evaluate the same frozen task and trajectory?
3. What evidence supports the rubric answers?
4. What was the verified trajectory outcome?
5. What uncertainty or disagreement remains?
6. Is the package ready for final JSON review and delivery?


Samples

Samples	These samples demonstrate the expected quality of a task prompt. A prompt is reviewed together with its frozen repository, environment, tests, golden solution, and verification assets. A good prompt does not need to explain the implementation, but it must define enough observable behavior and scope for an expert and an agent to determine what success means.

Good Samples

Task Rubric

Criteria specific to this task

	Index	Verifier Type
⠿	1	Prompt-specific rubric (binary YES/NO) LLM judge
⠿	2	Prompt-specific rubric (binary YES/NO) LLM judge
⠿	3	Prompt-specific rubric (binary YES/NO) LLM judge
⠿	4	Prompt-specific rubric (binary YES/NO) LLM judge
⠿	5	Prompt-specific rubric (binary YES/NO) LLM judge
⠿	6	Prompt-specific rubric (binary YES/NO) LLM judge

Other Annotations

[EXP] Shams-Ud-Deen

Dimension 0 · Goal completeness*

The gate, not a graded dimension.

PASS

Dimension 0 · reason*

Applied to a clean clone at the pinned commit, the patch builds, returns at 50ms under a 50ms deadline where the base tree take value. The success condition is met as written.

1 · Correctness*

Note: the full 1-5 bands for this dimension appear in neither the sp provisional until that guide arrives.

1 = core functional goal not achieved

1 · reason*

Measured rather than read. pkg/object/convert_context.go adds them while codecs.go covers the json and csv encoders. On a fr and go test ./pkg/vm/ exits 0 with the task's own gate applied. T than through a builtin; it returns in 182ms at 24 doublings becaus

2 · Architecture, Modularity & Trade-off Soundness*


Slack

Slack Guidelines & Usage

Slack is the primary communication channel for Imperium | SWE Trajectory QC Annotation — all major updates, task clarifications, and project-related discussions will be shared there. Please check Slack regularly, as this is where feedback, reminders, and important announcements will be posted.

Step 1: Join the Slack Workspace

Join our project's Slack workspace through the Okta setup on after signing your contract. Refer to the guide

Step 2: Set Your Display Name

Go to

1. Click your name (top right corner) & click "Account Settings"
2. Click the "Edit Profile" button
3. Click the "Edit" button in the "Personal Information" section
4. In the Display name field, input your updated display name
5. Click "Save" & the changes will propagate through to Slack (may take a bit)

Step 3: Join Required Channels

Upon joining Slack, you should be automatically added to the channels listed below.

Every writer and reviewer must be in the following channels:

🔔 **#anton-announcements-bf1**

- This channel is for official updates and organizational announcements from SPLs and EPMs that impact the entire project.
  🚫 **Please turn on all notifications for this channel**

🧰 **#anton-tooling-support-bf1**

- This channel is for troubleshooting issues related to project tools (e.g., Mercor Studio access, submission forms, or technical setup). If your issue persists for more than 24 hours without a response, DM Aahil Alion Slack.

🌐 **#anton-writers-5d4**

- **Upon integration into "Imperium | SWE Trajectory QC Annotation" slack please take a second to introduce yourself in this channel**
- This channel is a more casual setting for informal conversations, resource sharing, and building connections with teammates outside of work discussions.

Communication Guidelines

- Use **channels** for instances where the whole team can benefit from the shared answers.
    - General project queries → **#anton-general-bf1**
    - Tool or setup issues → **#anton-tooling-support-bf1**
- If you have not received a response within 24 hours, ping the relevant team member directly on Slack.
- Collaborate constructively with your reviewer when you receive feedback. Both of you share responsibility for delivering high-quality, on-time annotations.Please refrain from direct messaging project leadership (SPL, EPM, TPM) unless your question or concern hasn't been addressed in the channels.

Why We Use Channels

Posting in public channels ensures:

- Transparency and visibility for project managers.
- Creation of an evolving **FAQ** for common issues.
- Faster support — often, another team member might answer before staff do.

We **encourage open communication** and questions, it's better to ask in a channel than to stay uncertain. If a question is personal or confidential, feel free to contact any of the project leads.

Type of Issue	Contact
Tooling / setup issues	Post in **#anton-tooling-support-bf1**channel If your question isn't resolved promptly, please contact [EXP] Donna L on slack.
Annotation or reviewer-related issues	Post in **#anton-general-bf1**channel If your question isn't resolved promptly, please contact [EXP] Donna L on slack.
Billing / payment concerns	Contact [EXP] Donna L Aahil Ali on slack.
Personal or sensitive concerns	Contact [EXP] Donna L Aahil Ali on slack.



Etiquitte

- **Do not hoard tasks.** Please do not have multiple tasks in the **“Claimed”** state at the same time. If this continues to happen, we will:
    - Unassign the extra claimed tasks, and
    - Take further action if the behavior persists.
  - Hoarding tasks prevents others from having a fair opportunity to contribute to the project.

- **Keep communication professional.** If you have general questions, please post them in unless someone has specifically asked you to DM them. The leadership team receives a high volume of direct messages, so using the shared channel helps us respond more efficiently and ensures answers benefit everyone.

- **Prioritize quality over speed.** Your responses should meet our quality standards. Rushing to maximize throughput at the expense of quality may result in removal from the project. Please remember that task-based compensation only applies to tasks that are approved and delivered to the client. For example, completing 50 low-quality tasks that cannot be used will not result in payment for those tasks.

❓ FAQ

FAQ

Q: How does payment on the project work?

A: Writers are compensated on a per-task basis, while reviewers are compensated on an hourly basis.

Please note that payment is contingent on meeting the project's quality standards. Tasks that do not meet the required quality bar and are not suitable for client delivery will not be eligible for payment. However, reviewers may provide feedback and request revisions. As long as the requested feedback is addressed and the task ultimately meets the quality requirements, you will remain eligible for payment. Most tasks are expected to be one-shot at the desired quality standard.

Q: How can I become a reviewer?

A: Writers that consistently perform well (e.g., submit hard tasks that do not need reviews) and are strong community contributors (e.g., respond to questions in the) will be considered for reviewers.

Please do **not** reach out to Aahil, Donna, or another member of the leadership team to ask to be a reviewer. We will reach out if there is a need / believe you are a good fit.

Archive

Mercor Studio Guide

Studio Guide	Read below to walk through the studio workflow that is expected of you and find answers to common issues.

How this task works

Read the instructions below carefully.

What you do

You are not authoring the task. The task, both agent rollouts, and
other.

Work through the tabs in order. Everything below is **required** unle

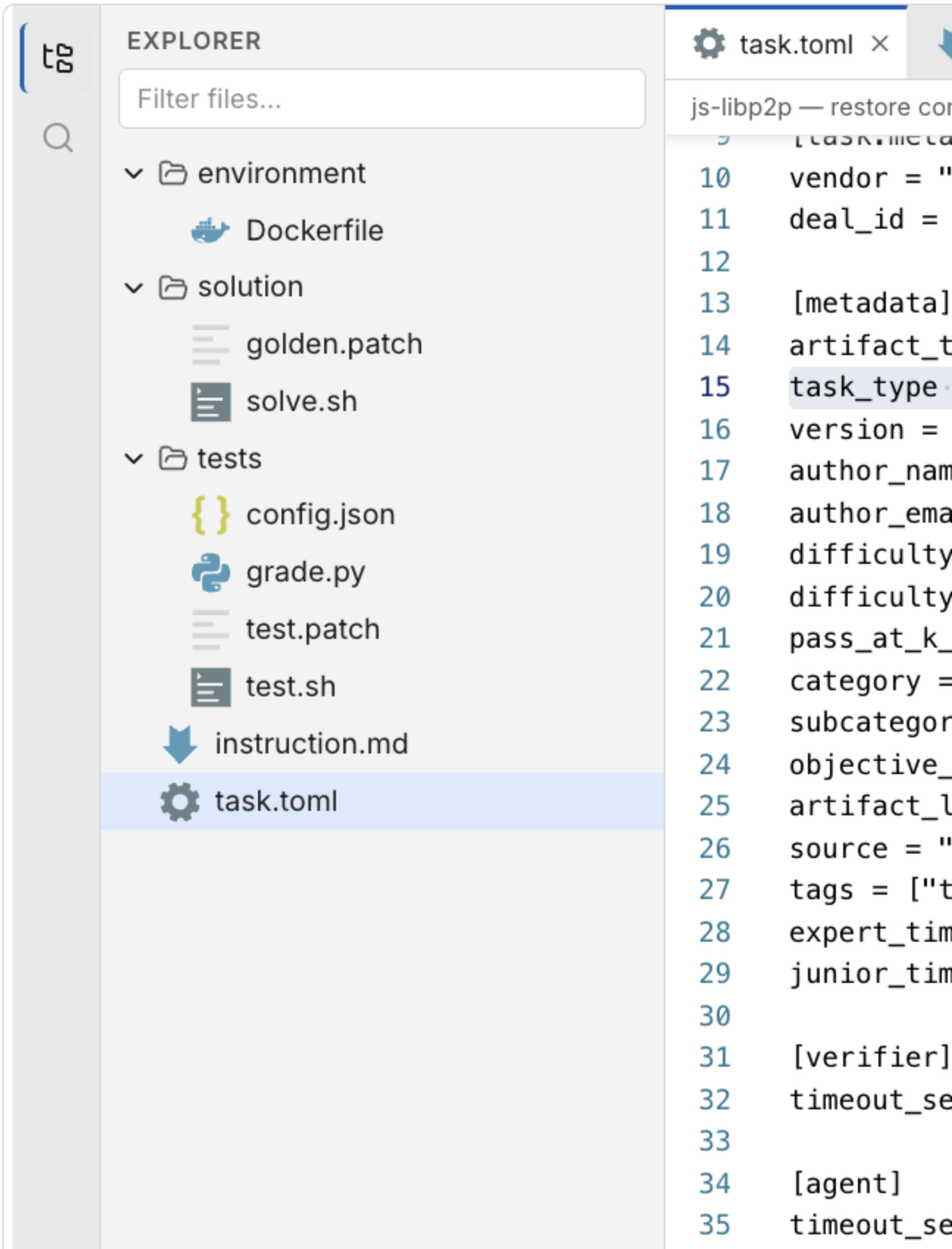
1. **Read the task (tab B).** Browse the bundle: `environment/`, `s
   metadata in `task.toml`.
2. **Score rollout A (tab C).** Read the agent solution, then the traj
   for the side. Dimension 0 is already filled from the recorded r
3. **Score rollout B (tab D).** The same, for the other rollout. Score
4. **Rubrics (tab E).** Write 8 to 15 binary questions where YES is a
   from the environment, or from common engineering knowled
   both sides with a reason citing evidence in that trajectory. *Ru*
5. **Pairwise comparison (tab F).** Give an overall preference of A
   scores.
6. **AutoQC and submit (tab G).** Run AutoQC, fix whatever it flag

Tab H is the independent reviewer's, not yours.

instruction.md

Task bundle

environment/, solution/, tests/, instruction.md and task.toml fo



Run two trajectories on the same harness, prompt, environment, and starting commit. At least one must be Gemini. Wait for both to reach Completed.

Score in this order:

1. Binary rubrics - you can answer these as each trajectory finishes.
2. Per trajectory: Dimension 0 (PASS / PARTIAL / FAIL, the gate), then the eight dimensions 1–5. Judge both the final answer *and* the step 0 → step n path, not just the diff.
3. Verdict: rank A vs. B and justify the comparison against specific evidence.

Done when: every dimension and every rubric item has a written reason. A reason that restates the score ("architecture was weak") isn't a reason - cite the step or the code.

2 · Trajectory A

The rollout as activity bands. Open a band to see the calls beneat

HARNESS

You are an AI assistant tasked with solving
output from previously executed comman

Format your response as JSON with the fo

{
"analysis": "Analyze the current state base
needs to be done?",
"plan": "Describe your plan for the next st
command to accomplish.",
"commands": [
{
"keystrokes": "ls -la\n",
"duration": 0.1
},
{
"keystrokes": "cd project\n",
"duration": 0.1
}
],
"task_complete": true
}

Required fields:

packages/utils/src/rate-limiter.ts

```
+ import { CodeError } from '@libp2p/interface'
+ import delay from 'delay'
+
+ export interface RateLimiterInit {
+   /**
+    * Number of points
+    *
+    * @default 4
+    */
+   points?: number
+
+   /**
+    * Per seconds
+    *
+    * @default 1
+    */
+   duration?: number
+
+   /**
+    * Block if consumed more than points in cur
+    *
+    * @default 0
+    */
+   blockDuration?: number
+
+   /**
+    * Execute allowed actions evenly over durat
+    *
+    * @default false
+    */
```

You can complete the binary rubrics as the trajectories finish. Then index by model A and model B to evaluate the model's final answer and the step 0 → step n trajectories.

Other Annotations

[EXP] Shams-Ud-Deen

Dimension 0 · Goal completeness*

The gate, not a graded dimension.

PASS

Dimension 0 · reason*

Applied to a clean clone at the pinned commit, the patch builds, a
bounded paths hold: string() returns at 50ms under a 50ms dead
A run that finishes inside its deadline still returns its full value. Th

1 · Correctness*

Note: the full 1-5 bands for this dimension appear in neither the spe
guide. These anchors are provisional until that guide arrives.

1 = core functional goal not achieved

5

1 · reason*

Measured rather than read. pkg/object/convert_context.go adds
string, sprintf and error through them while codecs.go covers the
cleanly, go build succeeds, go test ./... reports 24 packages ok, a
The one path outside the fix is a == a, which reaches List.Equals
182ms at 24 doublings because it allocates nothing, and the pron

Rubric

Define 8 to 15 binary questions where YES is always the good answer.

Task Rubric

Criteria specific to this task

	Index	Verifier Type	Cri
⠿	1	Prompt-specific rubric (binary YES/NO) LLM judge	Doe Wit cor mu wh
⠿	2	Prompt-specific rubric (binary YES/NO) LLM judge	Do still wit Err still
⠿	3	Prompt-specific rubric (binary YES/NO) LLM judge	Doe bui cas pro unb
⠿	4	Prompt-specific rubric (binary YES/NO) LLM judge	Doe dea for clo

F · Pairwise comparison

Compare the two sides head to head. The preference must a

Pairwise verdict

Pairwise Evaluation*

Select exactly one preference rating comparing Trajectory A

Select...

Comparative Rationale*

Explain your answer above.